openagent-api — Datasheet

> Reference document for building on top of, or integrating with, openagent-api.
> Intended audience: **openagent-frontend**, **openagent-infra**, **openagent-logger**, and any other service in the OpenAgent system that needs to understand what openagent-api is, what it owns, and how it talks to the rest of the system.

Quick Reference

Item	Value
Role	The Identity Gateway — auth + persona + SSE relay + fire-and-forget event emitter
Framework	FastAPI on uvicorn
Language	Python 3.11
Protocol in (frontend)	HTTP/1.1
Protocol out (openagent-infra)	HTTP/1.1 + Server-Sent Events (SSE consumer + re-emitter)
Protocol out (openagent-logger)	HTTP/1.1 (fire-and-forget JSON POST)
Host port	`8001`
Container port	`8001`
Auth in	`X-API-Key: OPENAGENT_API_KEY` (required on /chat and /health)
Auth out (openagent-infra)	`X-API-Key: INFRA_API_KEY` (attached on every upstream call)
Auth out (openagent-logger)	`X-API-Key: LOGGER_API_KEY` + HMAC-SHA256 signature keyed with `LOGGER_HMAC_SECRET` on every event payload
Inbound endpoints	`POST /chat` (SSE), `GET /health`
Outbound consumed	`POST /chat` (openagent-infra), `GET /health` (openagent-infra)
Outbound emitted	`POST /events` (openagent-logger, fire-and-forget; 5 events per successful /chat)
Backend dependency	openagent-infra (`OPENAGENT_INFRA_URL`, typically `:8002`)
Backend dependency	openagent-logger (`LOGGER_URL`, typically `:8003`)
Reasoning effort	Pass-through field on `/chat` (low / medium / high). No openagent-api default.
Session store	None — stateless across requests
Persistent store	None — bio.txt is read-only, baked into image
In-process state	LoggerClient queue (asyncio.Queue, default 1000) + background drain task

Item	Value
System prompt	`src/prompt/bio.txt`, baked into image
Version	1.0.0

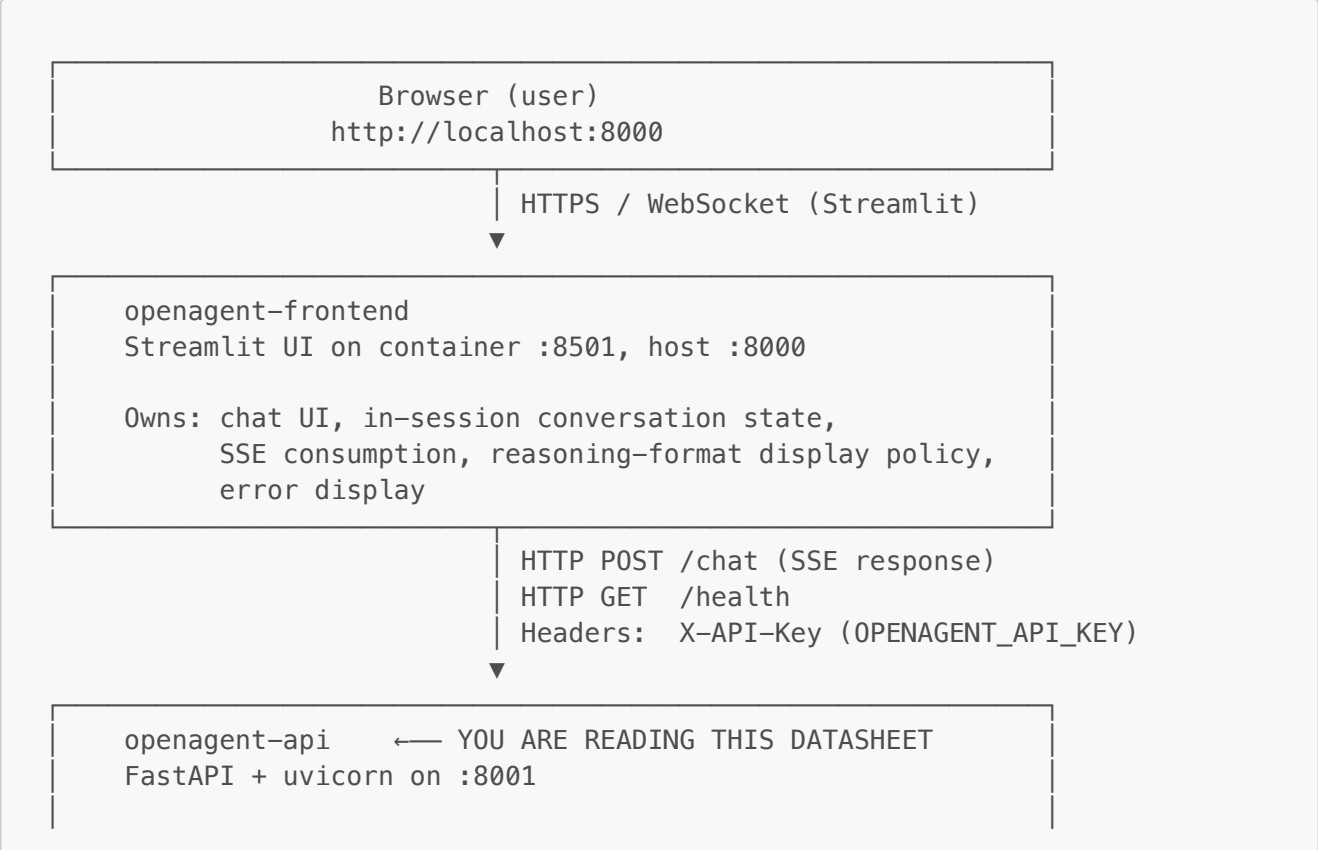
Overview

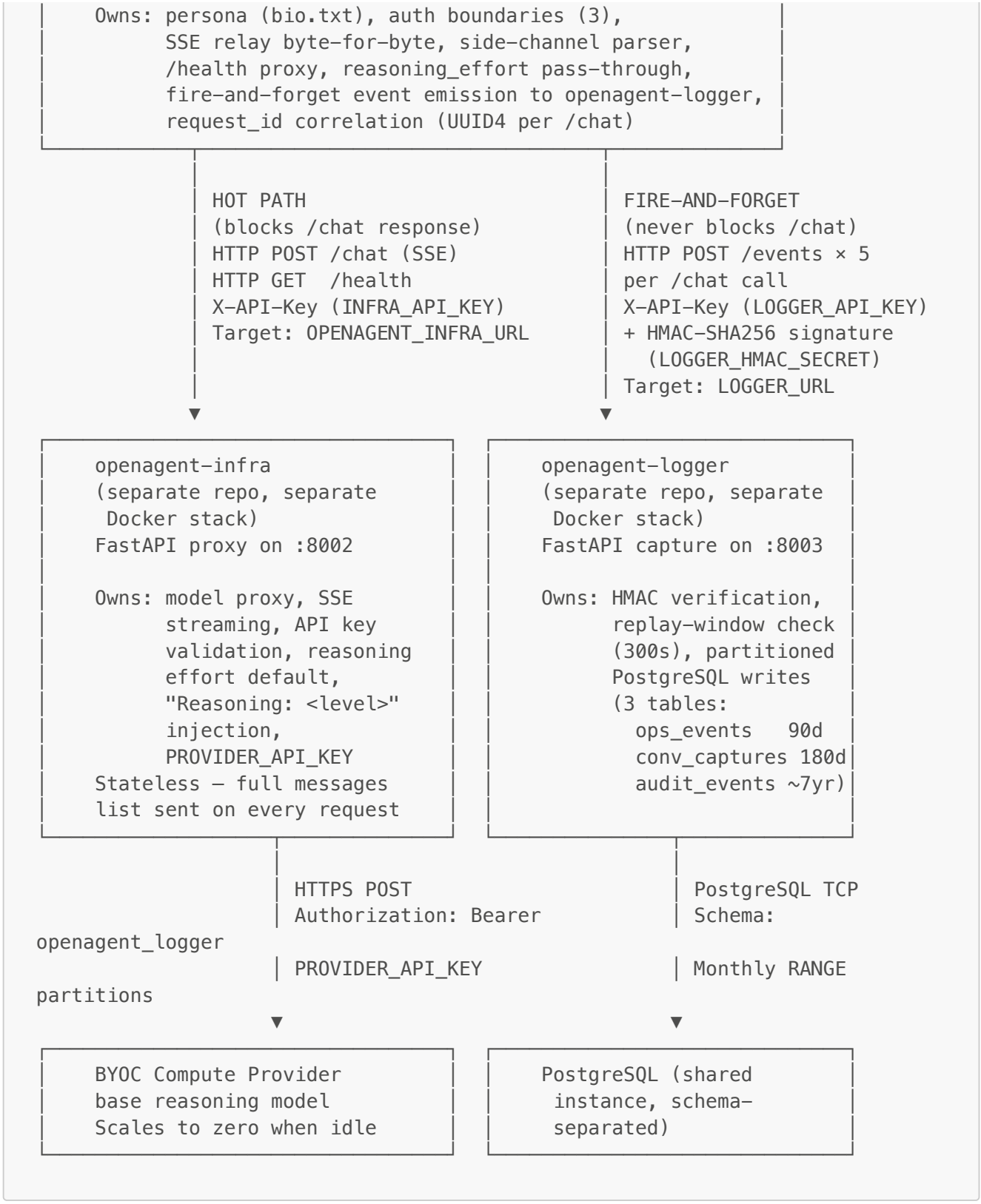
`openagent-api` is the **product-layer backend** of the OpenAgent system. It is a stateless (across requests) FastAPI gateway that owns four concerns and nothing else:

1. **The persona** — `src/prompt/bio.txt`, prepended as the first system message on every upstream call.
2. **The compartmentalized auth chain** — validates `OPENAGENT_API_KEY` inbound, attaches `INFRA_API_KEY` outbound to `openagent-infra`, attaches `LOGGER_API_KEY` and HMAC-signs with `LOGGER_HMAC_SECRET` outbound to `openagent-logger`. All secrets are independent values.
3. **The SSE relay** — opens a streaming POST to `openagent-infra` and pumps tokens byte-for-byte back to the frontend, with optional `reasoning_effort` pass-through and a side-channel parser that accumulates the visible answer for `conversation_capture`.
4. **The fire-and-forget event emitter** — every `/chat` call produces five structured events (`request_received`, `upstream_call`, `[upstream_error if applicable]`, `stream_complete`, `conversation_capture`) that `openagent-api` enqueues for a background drain task to deliver to `openagent-logger`. The `/chat` hot path never blocks on logger availability.

It is intentionally minimal. It does not load a model, run inference, host a database, authenticate users, validate session lifecycle, strip PII from captures, or interpret the model's reasoning format (the frontend's UX policy decision). It is an HTTP gateway with a side-channel observability sink, and nothing more.

Where This Service Fits





Port topology:

```
User → openagent-frontend (:8000) → openagent-api (:8001) → openagent-infra
(:8002) → BYOC provider
                                          │
                                          └→ openagent-logger (:8003) [fire-
and-forget sibling]
```

`openagent-api` is the **only** client of `openagent-infra`. It is the **only** thing `openagent-frontend` talks to over HTTP for inference. It is also the **only** emitter of events to `openagent-logger`.

Note on openagent-infra's two-model architecture: `openagent-infra` can serve a second model — a fast **nervous-system** control model — on a separate worker, selectable via an optional `model="nervous_system"` field on its `/chat` request. `openagent-api` **never sets that field**; every `/chat` call routes to the base model by default. The nervous-system route is available at the infra layer for callers that need a fast control model, and is outside `openagent-api`'s surface area today. The diagram shows the base-model path because that is the only path `openagent-api` exercises.

What This Service Owns

A strict list of responsibilities that live inside `openagent-api` and nowhere else.

1. The persona (system prompt)

The persona lives in `src/prompt/bio.txt` inside this repo. It is baked into the Docker image at build time (`COPY src/prompt/ /app/src/prompt/`) and loaded once at container startup via `load_identity()`. It is prepended as the first `{"role": "system", "content": <bio>}` message on every `/chat` call to `openagent-infra`.

- **`openagent-infra` has no knowledge of this file.** It receives the system message as part of the messages array and (only) appends `Reasoning: <level>` to it before forwarding to the provider. The persona text passes through unchanged.
- **`openagent-frontend` has no knowledge of this file.** It stops carrying any persona and stops sending system messages in the request body. If a frontend sends one anyway, `openagent-api` drops it with a warning log.
- **`openagent-logger` has no knowledge of this file.** The persona text is not emitted to the logger. `conversation_capture.input_text` carries USER messages only (not the system message), and `conversation_capture.output_text` carries the visible answer. The persona never leaves `openagent-api`.
- **No other service stores, duplicates, or overrides the persona.**
- **Changing the persona is a rebuild.** Editing `bio.txt` requires `docker-compose up -d --build` because it is baked into the image (or mount a volume over `/app/src/prompt` for live dev edits).

2. The inbound auth boundary (`OPENAGENT_API_KEY`)

`OPENAGENT_API_KEY` authorises `openagent-frontend` (or any future client) to talk to `openagent-api`. It is validated on every `/chat` and `/health` request via the `require_api_key` FastAPI dependency.

- Frontend sends it as the `X-API-Key` header.
- `openagent-api` compares it byte-for-byte against the value in its own environment.
- Mismatch returns `HTTP 401 {"detail": "Invalid or missing API key"}` — same shape as `openagent-infra`'s 401 so the frontend's emoji classifier surfaces 🔐 either way.
- This key never leaves `openagent-frontend` or `openagent-api`. `openagent-infra` never sees it. `openagent-logger` never sees it. The provider never sees it.

3. The outbound auth credential for openagent-infra (`INFRA_API_KEY`)

`INFRA_API_KEY` is the secret `openagent-api` uses to authenticate to `openagent-infra`. It is owned by the `InfraClient` (in `src/client/infra.py`), which constructs its internal `httpx.AsyncClient` at `start()` time with `headers={"X-API-Key": INFRA_API_KEY}` pre-attached so every outbound request carries it automatically.

- This key never leaves `openagent-api`'s environment. The frontend never sees it. The logger never sees it. The provider never sees it.
- It is **a different value** from `OPENAGENT_API_KEY`. Two independent secrets, separate blast radii.
- It must match `openagent-infra`'s `API_KEY` env var byte-for-byte.
- It is never logged, never echoed in responses, and never included in error messages.

The logger boundary uses a separate pair of secrets (`LOGGER_API_KEY` + `LOGGER_HMAC_SECRET`) following the same compartmentalization pattern — see "Outbound to openagent-logger" and the Security Model section.

4. The SSE relay

`openagent-api`'s `/chat` endpoint is the only stream-handling logic in the product layer. The implementation is an async generator (`sse_pump`) that:

- Opens a streaming POST to `OPENAGENT_INFRA_URL/chat` (via `infra_client.stream_chat(payload)`) with the messages list (system prompt prepended) and optionally `reasoning_effort`.
- Iterates `response.aiter_raw()` and yields each chunk straight through.
- Forwards `data:` events, `\n\n` separators, and the `[DONE]` sentinel byte-for-byte.
- Forwards each event payload — a JSON-encoded OpenAI ChatCompletion chunk with `delta.reasoning` (chain-of-thought) and `delta.content` (visible answer) tokens — without decoding or interpreting the JSON on the relay path.
- **Runs a side-channel parser** in parallel with the byte-for-byte relay. Each yielded chunk is appended to an `event_buffer`, split on `\n\n`, and parsed as JSON to extract `choices[0].delta.content` tokens into an `output_text` accumulator. This accumulator feeds the eventual `conversation_capture` event. **The yield to the frontend happens BEFORE the parse in every loop iteration**, so the parse adds zero latency to the user-visible stream. Parse failures are silently tolerated.
- Detects mid-stream client disconnects via `await http_request.is_disconnected()` and abandons the upstream connection so the provider stops generating.
- Surfaces non-200 upstream responses as in-band SSE error events: `data: [ERROR upstream_status=503]\n\n` followed by `data: [DONE]\n\n`. An `upstream_error` event is also emitted to the logger before yielding the in-band error.

No other service consumes `openagent-infra`'s SSE stream directly. `openagent-api` is the only client.

5. The `reasoning_effort` pass-through

`reasoning_effort` is an optional field on the inbound `/chat` request body. Three accepted values: `low`, `medium`, `high`. Pydantic validates the value if present; an invalid value produces HTTP 422 before any upstream call.

The behaviour is **pure pass-through**:

- When the frontend sends a value, `openagent-api` includes it in the upstream payload.

- When the frontend omits the field, `openagent-api` also omits it — `openagent-infra` applies its own server-side default (controlled by `openagent-infra`'s `REASONING_EFFORT` env var).

`openagent-api` holds **no default of its own** for this field. The setting has one source of truth at the upstream layer.

Note: `reasoning_effort` appears on the `request_received`, `upstream_call`, and `conversation_capture` events emitted to the logger as well, so downstream analysis can correlate latency, output length, and reasoning effort.

6. The `/health` proxy

`openagent-api` exposes its own `/health` that proxies `openagent-infra`'s `/health`. The frontend polls here every 3 seconds during cold start to gate the chat input. Response shape:

```
{
  "status": "ok" | "loading" | "unreachable",
  "openagent_api": {"version": "1.0.0", "identity_loaded": true},
  "openagent_infra": {
    "url": "http://...:8002",
    "status": "ok" | "loading" | "unreachable",
    "raw": {}
  }
}
```

The top-level `status` reflects the worst of `(openagent-api state, openagent-infra state)`. The frontend's gate-open loop reads only this field.

Status mapping (openagent-infra → openagent-api):

upstream `status`	openagent-api `status`	Meaning
`ok`	`ok`	Provider worker warm and serving
`degraded`	`loading`	Provider worker cold-starting
`loading`	`loading`	(Kept for backward compat)
anything else	`unreachable`	infra responded but with an unknown state
no response	`unreachable`	infra is down or unreachable from openagent-api

The `degraded` → `loading` translation: `openagent-infra` reports `degraded` when its FastAPI proxy is healthy but its provider worker is cold-starting (serverless workers scale to zero when idle). Semantically that's the same condition the frontend calls `loading` — model isn't ready, hold the chat input closed. `openagent-api` translates the field so the frontend reads consistent semantics. This translation lives inside `InfraClient.check_health()`, which returns `Tuple[str, Optional[Dict]]` (translated status + raw upstream body) and never raises; any failure to obtain a healthy response translates internally to `("unreachable", None)`.

Note on the upstream raw body: the `raw` field carries `openagent-infra`'s `/health` body unchanged. `openagent-api` **forwards it verbatim** — it does not parse, rename, or drop keys. The top-level `status` semantics are stable, so the mapping table works regardless of the exact `raw` shape; `raw` is informational only.

The endpoint requires the same `X-API-Key` as `/chat` because it reveals operational state (upstream URL, version, worker readiness) that should not be public. `/health` does NOT include `openagent-logger`'s status and does NOT emit events to the logger: gate-open is polled every 3 seconds, so coupling it to logger availability would defeat fire-and-forget, and emitting per-poll events would flood the logger with no operational value.

7. Frontend-supplied system message rejection

If a `system` message is sent in a `/chat` request body — by an out-of-date frontend, a tampered client, or a misconfigured tool — `openagent-api` drops it server-side and logs a warning. The canonical persona is `bio.txt` and only `bio.txt`. This is defense in depth.

8. Error taxonomy mapping (gateway layer)

`openagent-api` maps upstream error conditions to the HTTP status codes the frontend's emoji classifier understands:

Frontend emoji	Status	Trigger from openagent-api
🔌	502	Cannot reach openagent-infra (TCP connect failed, network unreachable)
⏳	503	openagent-infra reports `degraded` / `loading` (provider worker cold-start)
🔌	504	Upstream read timeout during generation
🔐	401	Inbound `X-API-Key` missing or wrong
⚠	400	Empty messages list / no user message after server-side filtering
⚠	422	Malformed request body / invalid `reasoning_effort` value
❌	500	Anything not matched above

9. Fire-and-forget event emission to openagent-logger

`openagent-api` is the only emitter of events to `openagent-logger`. Five events are emitted per successful `/chat` call (four on the failure path); each carries a shared `request_id` (UUID4) so downstream queries can reconstruct a `/chat`'s full event timeline.

The five emission points within `/chat`:

- `request_received` — at ingress, after auth passes
- `upstream_call` — immediately before opening the infra stream
- `upstream_error` — in every exception handler in the SSE pump
- `stream_complete` — after the SSE pump finishes (success or client_disconnect)
- `conversation_capture` — after a successful stream_complete only

The emission is fire-and-forget by design: emit methods are synchronous, do no I/O, return in microseconds, and enqueue onto an in-process `asyncio.Queue`. A background `asyncio.Task` drains the queue and POSTs to the logger; failures result in WARNING logs and dropped events, with no impact on `/chat`. Queue overflow is handled by drop-oldest.

The full wire contract is documented in the Outbound to openagent-logger section below.

What This Service Does NOT Own

- **Model serving / inference** → the BYOC compute provider (proxied by `openagent-infra`)
- **Provider authentication (`PROVIDER_API_KEY`)** → `openagent-infra`
- **Model API key validation** → `openagent-infra`
- **`Reasoning: <level>` injection into the system message** → `openagent-infra`
- **Reasoning effort default value** → `openagent-infra` (its `REASONING_EFFORT` env var)
- **Event storage, partitioning, retention** → `openagent-logger`
- **Reasoning-format display policy** → `openagent-frontend`
- **In-session conversation state** → `openagent-frontend`
- **Chat UI rendering** → `openagent-frontend`

Not implemented:

- **PII stripping / sanitisation of conversation_captures** → not implemented; captures are stored raw.
- **Session lifecycle and session-id validation** → not implemented; `session_id` on every event is emitted as `null`.
- **Per-user identity / authentication** → not implemented; `user_id` is emitted as `null` on every event.
- **Persistent conversation history** → not implemented.
- **Reasoning chain capture in `conversation_capture.output_text`** → not captured. `output_text` is `delta.content` only.
- **Durable retry of failed event submissions** → not implemented; `openagent-api` drops events on POST failure with no retry.
- **Rate limiting** → not implemented (belongs at a reverse proxy if ever needed).
- **Multi-tenancy** → not supported.

Inbound HTTP Contracts (provided)

`POST /chat`

Forward a message list, optionally specify reasoning effort, get back an SSE stream. Authenticated.

Request:

```
POST /chat HTTP/1.1
Host: openagent-api:8001
Content-Type: application/json
X-API-Key: <OPENAGENT_API_KEY>

{
  "messages": [
    {"role": "user",      "content": "..."},
    {"role": "assistant", "content": "..."},
    {"role": "user",      "content": "..."}
  ],
  "reasoning_effort": "medium"
}
```

Field	Type	Required	Description

Field	Type	Required	Description
`messages`	array	Yes	Non-empty list. Must contain at least one `user` message after server-side filtering.
`messages[].role`	string	Yes	One of `user`, `assistant`, `system`. `system` messages are dropped server-side.
`messages[].content`	string	Yes	Non-empty message text.
`reasoning_effort`	string	No	One of `low`, `medium`, `high`. Forwarded upstream when present; omitted when absent so `openagent-infra` applies its default. Validated by Pydantic — invalid values produce HTTP 422 before any upstream call.

Header:

Header	Required	Description
`X-API-Key`	Yes	Must match `OPENAGENT_API_KEY` byte-for-byte.

Response: `text/event-stream`

```
HTTP/1.1 200 OK
Content-Type: text/event-stream; charset=utf-8
Cache-Control: no-cache
X-Accel-Buffering: no
Connection: keep-alive

data: {"id":"chatcmpl-...","object":"chat.completion.chunk","choices":
[{"index":0,"delta":{"role":"assistant","content":""},"finish_reason":null}]}

data: {"id":"chatcmpl-...","object":"chat.completion.chunk","choices":
[{"index":0,"delta":{"reasoning":"User"},"finish_reason":null}]}

...  (more reasoning tokens — chain-of-thought)

data: {"id":"chatcmpl-...","object":"chat.completion.chunk","choices":
[{"index":0,"delta":{"content":"Hello"},"finish_reason":null}]}

...  (more content tokens — visible answer)

data: {"id":"chatcmpl-...","object":"chat.completion.chunk","choices":
[{"index":0,"delta":{},"finish_reason":"stop"}]}

data: [DONE]
```

The stream is `openagent-infra`'s stream re-emitted byte-for-byte. Each event payload is a JSON-encoded OpenAI ChatCompletion chunk. Chain-of-thought tokens stream first inside `choices[0].delta.reasoning`, then visible answer tokens inside `choices[0].delta.content`, then a final empty-delta chunk with `finish_reason: "stop"`, then `[DONE]`. `openagent-api` does not decode any of this on the relay path — the frontend's parser routes the two streams. A side-channel parser inside `openagent-api` parses each event in parallel to extract `delta.content` for the `conversation_capture`;

that parse is downstream of the yield and adds zero latency. Mid-stream upstream failures are surfaced as in-band events: `data: [ERROR upstream=...]\n\n` followed by `data: [DONE]\n\n`.

Side effects: Each call triggers five fire-and-forget event emissions to `openagent-logger` (four on the failure path):

1. `request_received` at ingress (after auth)
2. `upstream_call` before opening the infra stream
3. `upstream_error` on every exception path (instead of `stream_complete` and `conversation_capture`)
4. `stream_complete` after the stream finishes (success or client_disconnect)
5. `conversation_capture` after a successful `stream_complete` only (not for client_disconnect)

All emissions share the same `request_id` (UUID4); `session_id` and `user_id` are emitted as `null`. See Outbound to openagent-logger.

Generation timing (warm path):

Scenario	reasoning_effort	Approximate duration
Simple greeting	`low`	5–15 seconds
Short factual question	`medium`	15–45 seconds
Complex reasoning task	`high`	1–3 minutes

Cold path: add the provider's serverless worker spin-up time for the first request after the worker has scaled to zero.

Error responses:

Status	Trigger	Body
`400`	Empty messages list, or no `user` message after filtering	`{"detail": "..."}`
`401`	`X-API-Key` missing or wrong	`{"detail": "Invalid or missing API key"}`
`422`	Request body fails Pydantic validation (including invalid `reasoning_effort`)	FastAPI default validation error
`500`	openagent-api internal error before stream open	`{"detail": "..."}`

`GET /health`

Proxied health check. Authenticated.

Request:

```
GET /health HTTP/1.1
X-API-Key: <OPENAGENT_API_KEY>
```

Response (always HTTP 200):

```
{
  "status": "ok" | "loading" | "unreachable",
  "openagent_api": {"version": "1.0.0", "identity_loaded": true},
  "openagent_infra": {
    "url": "http://openagent-infra:8002",
    "status": "ok" | "loading" | "unreachable",
    "raw": {}
  }
}
```

Status semantics (top-level):

- `ok` — openagent-api is up AND openagent-infra reports `ok` AND its provider worker is warm
- `loading` — openagent-api is up but openagent-infra reports `degraded` or `loading` (provider worker cold-starting)
- `unreachable` — openagent-api is up but cannot reach openagent-infra

`/health` does NOT include `openagent-logger`'s status. To check the logger directly, query its own `/health` at `LOGGER_URL/health` with `X-API-Key: LOGGER_API_KEY`.

Outbound HTTP Contracts (consumed)

`openagent-api` is a client of these endpoints, in a request/response (streaming) pattern. Full specs live in `openagent-infra`'s datasheet.

The `openagent-logger` boundary is NOT a consumed contract in this sense; it is a fire-and-forget emission, documented in Outbound to openagent-logger.

`POST {OPENAGENT_INFRA_URL}/chat` — consumed

Request:

```
POST /chat HTTP/1.1
Content-Type: application/json
X-API-Key: <INFRA_API_KEY>

{
  "messages": [
    {"role": "system",    "content": "<bio.txt>"},
    {"role": "user",      "content": "..."},
    {"role": "assistant", "content": "..."}
  ],
  "reasoning_effort": "medium"
}
```

The `system` message is the `bio.txt` content prepended by `openagent-api`. The remaining messages are the frontend's payload after `system` filtering. `reasoning_effort` is included only when the frontend sent one. This POST is issued via `InfraClient.stream_chat(payload)` in `src/client/infra.py`.

Note on the `model` field: `openagent-infra`'s `/chat` accepts an optional `model` field (`"base"` or `"nervous_system"`) selecting between its two workers — the base reasoning model and a fast nervous-system control model. `openagent-api` **never sets this field**; every request omits it, which routes the call to the base model by default. The nervous-system route is available at the infra layer for callers that need it and is outside `openagent-api`'s surface area.

Response: `text/event-stream`. Each event payload is a JSON-encoded OpenAI ChatCompletion chunk. Reasoning tokens appear in `choices[0].delta.reasoning`, visible answer tokens in `choices[0].delta.content`, terminating with an empty-delta chunk (`finish_reason: "stop"`) followed by `data: [DONE]`. `openagent-api` consumes this stream byte-for-byte on the relay path and runs a side-channel parser in parallel to extract content tokens for the `conversation_capture`.

Timeout handling:

- Connect timeout: 10 seconds (`UPSTREAM_CONNECT_TIMEOUT`).
- Read timeout: unbounded by default (`UPSTREAM_READ_TIMEOUT=none`) — accommodates provider cold-start and `high` reasoning effort generations.

`GET {OPENAGENT_INFRA_URL}/health` — consumed

Request:

```
GET /health HTTP/1.1
X-API-Key: <INFRA_API_KEY>
```

(`openagent-api` sends its key on every outbound call, including `/health`. `openagent-infra` does not require auth on `/health`, but the header is harmless.)

Response: Always HTTP 200, status in the body. The top-level `status` (`"ok"` means the base worker is reachable, `"degraded"` means it's cold-starting) is what `openagent-api` translates into its own status vocabulary via `InfraClient.check_health()`. `openagent-api` forwards the entire raw body verbatim under `openagent_infra.raw` in its own `/health` response — it does not parse, rename, or drop keys.

Timeout handling: 5-second connect/read (`HEALTH_TIMEOUT`). `check_health()` catches all errors internally and never raises — any httpx error, non-200 status, JSON-parse error, or non-dict body translates to `("unreachable", None)`.

Outbound to openagent-logger

> **Authoritative reference** for the openagent-api → openagent-logger boundary, intended for downstream services (openagent-logger, auditors) that need to verify what openagent-api emits, when, and how. The protocol described here is implemented in `src/client/logger.py` and matched byte-for-byte by openagent-logger's ingress validator.

Pattern: fire-and-forget

`openagent-api` emits events into an in-process `asyncio.Queue` and returns from each emit method in microseconds. A background `asyncio.Task` drains the queue and POSTs to `openagent-logger`. The `/chat` hot path is never blocked on logger availability. If the logger is unreachable or slow, events queue up (eventually dropping per the overflow policy) while `/chat` continues to serve normally.

This is the OPPOSITE of the `openagent-api` → `openagent-infra` pattern. The infra boundary is fully synchronous (block `/chat` until upstream responds, then stream byte-for-byte). The logger boundary is fully asynchronous (emit and forget). The two patterns reflect their different roles: infra produces the response, the logger captures the side-effect data.

Outbound endpoint

```
POST {LOGGER_URL}/events
Content-Type: application/json
X-API-Key: <LOGGER_API_KEY>
```

All events go to the single `/events` endpoint regardless of `event_type`; `openagent-logger` routes them to the appropriate storage table internally.

Event envelope (JSON body)

```
{
  "request_id": "<uuid4 dashed>",
  "session_id": null,
  "user_id": null,
  "source_service": "openagent-api",
  "client_timestamp": "<ISO 8601 UTC with Z>",
  "event_type": "<event-type string, see below>",
  "payload": {},
  "hmac_signature": "<lowercase hex, 64 chars>"
}
```

Field	Type	Required	Description
`request_id`	string (UUID4 with dashes, 36 chars)	Yes	Generated at `/chat` ingress via `str(uuid.uuid4())`, e.g. `cf8108cb-958d-4f11-a265-d885ac52415d`. All five events for one `/chat` call share the same request_id so they can be joined post-hoc by querying the logger's database.
`session_id`	string or null	Yes	Reserved correlation field. `openagent-api` emits `null` — there is no session tracking in the current stack. A caller that tracks sessions could populate it without a schema change; the logger never validates it.
`user_id`	string or null	Yes	Reserved correlation field. Emitted as `null` — there is no per-user identity in the current stack.
`source_service`	string	Yes	Always `"openagent-api"` for events emitted from this service. The logger uses this to scope queries by emitter.

Field	Type	Required	Description
`client_timestamp`	string (ISO 8601 UTC)	Yes	UTC timestamp at the moment `openagent-api` enqueued the event. Format: `2026-05-15T19:53:14.123456Z` (always `Z`, microsecond precision). The logger uses this for the replay-window check (events older than 300 seconds are rejected).
`event_type`	string	Yes	One of `request_received`, `upstream_call`, `upstream_error`, `stream_complete`, `conversation_capture`.
`payload`	object	Yes	Event-type-specific fields; see below. Always a JSON object.
`hmac_signature`	string (lowercase hex, 64 chars)	Yes	HMAC-SHA256 of the canonical string (see below), keyed with `LOGGER_HMAC_SECRET`. Verifiable offline by anyone with the secret.

Event types and routing

event_type	Routed to (openagent-logger table)	Retention
`request_received`	`ops_events`	90 days
`upstream_call`	`ops_events`	90 days
`upstream_error`	`ops_events`	90 days
`stream_complete`	`ops_events`	90 days
`conversation_capture`	`conversation_captures`	180 days

Routing is the logger's concern. `openagent-api` just sets the `event_type` value. The `audit_events` table (~7yr retention) in the logger is not currently written by `openagent-api`.

Per-event payload shapes

`request_received`

Emitted at `/chat` ingress, after auth passes, before bio.txt prepending.

```
{ "messages_count": 7, "reasoning_effort": "medium" }
```

Field	Type	Description
`messages_count`	int	Number of messages in the inbound body (before server-side filtering).
`reasoning_effort`	string or null	The `reasoning_effort` from the inbound body verbatim, or null.

`upstream_call`

Emitted in `sse_pump` immediately before opening the streaming POST to `openagent-infra`.

```
{ "upstream_url": "http://host.docker.internal:8002/chat",
"reasoning_effort": "medium" }
```

Field	Type	Description
`upstream_url`	string	Full URL about to be called, typically `f"{OPENAGENT_INFRA_URL}/chat"`.
`reasoning_effort`	string or null	Same as in `request_received`.

`upstream_error`

Emitted in every exception handler in the SSE pump.

```
{ "error_type": "ConnectTimeout", "status_code": null }
```

Field	Type	Description
`error_type`	string	Exception class name as `type(exc).__name__` (e.g. `ConnectTimeout`, `ReadTimeout`, `ConnectError`, `HTTPStatusError`, `RemoteProtocolError`, `Exception`).
`status_code`	int or null	HTTP status code if applicable; null otherwise.

When `upstream_error` is emitted, no `stream_complete` and no `conversation_capture` follow for this `request_id`. The chain for a failed call is: `request_received` → `upstream_call` → `upstream_error`.

`stream_complete`

Emitted after the SSE pump finishes. Two outcomes: clean success, or mid-stream client disconnect.

```
{ "bytes_relayed": 8421, "latency_ms": 1735, "outcome": "success" }
```

Field	Type	Description
`bytes_relayed`	int	Total bytes pumped from infra to the frontend.
`latency_ms`	int	Milliseconds from `/chat` ingress to emission.
`outcome`	string	`"success"` (clean completion through `[DONE]`) or `"client_disconnect"`.

For `outcome=client_disconnect`, no `conversation_capture` follows (the captured response would be partial).

`conversation_capture`

Emitted only after a `stream_complete` with `outcome=success`. The full conversation snapshot, stored for operational record-keeping and offline integrity verification.

```
{
  "input_text": "User question text concatenated from all user-role
messages.",
  "output_text": "The visible answer, concatenated from delta.content
tokens.",
  "input_hash": "<sha256 hex of input_text>",
  "output_hash": "<sha256 hex of output_text>",
  "model_used": "base-model",
  "reasoning_effort": "medium",
  "latency_ms": 1735,
  "input_tokens": null,
  "output_tokens": null
}
```

Field	Type	Description
`input_text`	string	All user-role messages from the inbound body, joined by `\n`. System messages are excluded (the persona is owned by `openagent-api`); assistant messages are excluded too — only fresh user input is the "input" for this turn.
`output_text`	string	The visible answer, accumulated from `choices[0].delta.content` tokens via the side-channel parser. The reasoning chain (`delta.reasoning`) is NOT included — see Design Decisions.
`input_hash`	string (sha256 hex)	`hashlib.sha256(input_text.encode("utf-8")).hexdigest()`. Lets consumers detect duplicates without storing the text twice.
`output_hash`	string (sha256 hex)	Same algorithm applied to `output_text`.
`model_used`	string	The model identifier the provider returned (from `choices[0].model` in the stream).
`reasoning_effort`	string	The effective value for this call (the frontend's value verbatim, or the server-side default).
`latency_ms`	int	Same as in `stream_complete` for the same `request_id`.
`input_tokens`	int or null	From `usage.prompt_tokens` if the stream provides a final usage chunk; null otherwise.

Field	Type	Description
`output_tokens`	int or null	From `usage.completion_tokens` if available; null otherwise.

Canonical string and HMAC signature

The `hmac_signature` in every envelope is computed as follows, implemented byte-for-byte in `src/client/logger.py` and reversed on ingress by `openagent-logger`.

Step 1 — Canonical payload JSON.

```
canonical_payload_json = json.dumps(
    payload,
    sort_keys=True,
    separators=(",", ":"),
    default=str,
    ensure_ascii=False,
)
```

The four `json.dumps` keyword arguments are non-negotiable; changing any breaks the signature on both sides:

kwarg	Value	Why
`sort_keys`	`True`	Key ordering must be byte-stable so two services serializing the same dict produce identical bytes.
`separators`	`(",", ":")`	Removes ALL whitespace between fields, so re-serialization can't introduce nondeterminism.
`default`	`str`	Safety net for non-JSON-serialisable values.
`ensure_ascii`	`False`	Allows Unicode through unencoded; both sides encode UTF-8 before hashing.

Step 2 — Payload hash.

```
payload_hash = hashlib.sha256(canonical_payload_json.encode("utf-
8")).hexdigest()
```

Step 3 — Canonical string.

```
canonical_string = f"{request_id}|{client_timestamp}|{event_type}|
{payload_hash}"
```

Four pipe-separated components in this exact order: `request_id`, `client_timestamp`, `event_type`, `payload_hash`. Pipe `|` (U+007C) is the separator, no surrounding whitespace.

Step 4 — HMAC.

```
hmac_signature = hmac.new(
    LOGGER_HMAC_SECRET.encode("utf-8"),
    canonical_string.encode("utf-8"),
    hashlib.sha256,
).hexdigest()
```

`openagent-logger` reverses this exact computation on ingress and compares (constant-time, via `hmac.compare_digest`) against the supplied `hmac_signature`. Mismatch returns HTTP 401 (`Invalid HMAC signature`) and the event is rejected.

Verification property. Because the signature is computed over (request_id, client_timestamp, event_type, sha256(canonical_payload_json)) — NOT the wire envelope — it survives lossless re-encoding. An auditor with the stored payload and `LOGGER_HMAC_SECRET` can re-verify any stored event offline, without trusting any transport claim. This is why `LOGGER_HMAC_SECRET` exists as a separate secret from `LOGGER_API_KEY` (transport) — see Security Model.

Queue and background task

The `LoggerClient` in `src/client/logger.py` holds three pieces of runtime state:

1. **`_http_client`** — `httpx.AsyncClient` configured with `base_url=LOGGER_URL`, `headers={"X-API-Key": LOGGER_API_KEY}`, and short timeouts (5s connect, 10s read — fire-and-forget should fail fast). A SEPARATE instance from the `InfraClient`'s internal httpx client; the two boundaries have different headers, base URLs, and timeout characteristics.
2. **`_queue`** — `asyncio.Queue` with capacity `OPENAGENT_LOGGER_QUEUE_MAX_SIZE` (env var, default 1000).
3. **`_drain_task`** — `asyncio.Task` running `_drain_loop()` until cancelled.

Startup (FastAPI lifespan): `LoggerClient(...)` is constructed and `await logger_client.start()` creates `_http_client` and launches `_drain_task`. No connectivity probe — `openagent-api` boots even if the logger is unreachable.

Runtime (during `/chat`): emit methods are synchronous, do no I/O, return in microseconds. Each emit call builds the envelope, computes the signature, and `put_nowait`s onto the queue. If the queue is full, it catches `asyncio.QueueFull`, pops the oldest event, logs a WARNING, calls `task_done()` to keep accounting clean, then enqueues the new envelope.

Drain task (`_drain_loop`): pulls envelopes, POSTs to `{LOGGER_URL}/events`. On non-2xx or network error, logs a single WARNING and returns — no retry, no re-enqueue. The drained event is gone.

Shutdown (FastAPI lifespan): `await logger_client.stop()` waits up to 5 seconds for `_queue.join()`, then cancels `_drain_task` and closes `_http_client`. This happens BEFORE the `InfraClient` stops, so any events still in the queue get one last chance to land.

Failure modes

Failure	Detection	openagent-api behavior	Operator observability

Failure	Detection	openagent-api behavior	Operator observability
logger unreachable (TCP/DNS fail)	`httpx.ConnectError`	Drop event, log WARNING	`WARNING | LoggerClient POST failed: ConnectError to {url}`
logger slow (timeout)	`httpx.ReadTimeout`	Drop event, log WARNING	`WARNING | LoggerClient POST failed: ReadTimeout to {url}`
logger 401 (transport key mismatch)	Non-2xx, "Invalid or missing API key"	Drop event, log WARNING	fix by aligning `LOGGER_API_KEY` in both `.env` files
logger 401 (HMAC mismatch)	Non-2xx, "Invalid HMAC signature"	Drop event, log WARNING	fix by aligning `LOGGER_HMAC_SECRET` in both `.env` files
logger 422 (malformed envelope)	Non-2xx	Drop event, log WARNING	should not happen if both sides match the wire contract
Queue full (sustained outage)	`asyncio.QueueFull`	Pop oldest, push new, log WARNING	`WARNING | LoggerClient queue full (max=N); dropped oldest event`

Throughout all failure modes, `/chat` is completely unaffected. The frontend cannot tell whether the logger is up. That is the design.

What this section does NOT cover

- **PII stripping**: `openagent-api` does none. `input_text`/`output_text` are stored raw.
- **Session-id / user-id population**: always `null`; no session or user tracking in the current stack.
- **Reasoning chain capture**: `output_text` is `delta.content` only; `delta.reasoning` is not captured.
- **Durable outbox / retry**: events lost during a logger outage are lost permanently.

State Model

Per-process state (populated at startup)

Symbol	Type	Lifetime	Purpose
`identity`	`str`	Process	`bio.txt` content, prepended as the system message on every `/chat`.

Symbol	Type	Lifetime	Purpose
`infra_client`	`InfraClient`	Process	Encapsulates the outbound boundary to `openagent-infra`. Owns an internal `httpx.AsyncClient` with `base_url=OPENAGENT_INFRA_URL`, `X-API-Key: INFRA_API_KEY` pre-attached, connect 10s, read unbounded by default, write 10s, pool 5s. Exposes `start()`, `stop()`, `aclose()`, `stream_chat(payload)` (returns httpx's streaming context manager so `sse_pump`'s `async with` is unchanged), and `check_health()` (returns `Tuple[str, Optional[Dict]]`, never raises). Initialized at lifespan startup, stopped at shutdown.
`logger_client`	`LoggerClient`	Process	Fire-and-forget emitter for `openagent-logger`. Holds a SEPARATE `httpx.AsyncClient` with `X-API-Key: LOGGER_API_KEY` pre-attached, `base_url=LOGGER_URL`, connect 5s, read 10s. Initialized at lifespan startup, stopped at shutdown.
`logger_client._queue` + `logger_client._drain_task`	`asyncio.Queue` + `asyncio.Task`	Process (internal)	In-memory pending-event buffer + background drain task. Capacity `OPENAGENT_LOGGER_QUEUE_MAX_SIZE` (default 1000), drop-oldest overflow. Drain task POSTs to the logger; failures logged as WARNING and event dropped with no retry.

Per-request state

None retained. The gateway is **stateless across requests**. Each `/chat` generates a fresh `request_id` (UUID4) to thread the emitted events together, but the request_id is not persisted in `openagent-api` — it lives only on the events the logger receives.

Persistent state

None. `bio.txt` is read-only and baked into the image. Restarting the container loses no data because there is none — including any pending events in the LoggerClient queue. This is an acknowledged limitation.

Configuration

All runtime configuration is loaded from `.env` at the repository root via `python-dotenv` and `docker-compose`'s `env_file:` directive.

Required

Variable	Type	Description
`OPENAGENT_API_KEY`	string	Inbound secret. openagent-api refuses to start if unset.
`INFRA_API_KEY`	string	Outbound secret to openagent-infra. Refuses to start if unset. Must match openagent-infra's `API_KEY` byte-for-byte.
`OPENAGENT_INFRA_URL`	string	Base URL of openagent-infra. No trailing slash.
`LOGGER_URL`	string	Base URL of openagent-logger. No trailing slash. Refuses to start if unset.
`LOGGER_API_KEY`	string	Outbound transport secret to openagent-logger. Must match its `LOGGER_API_KEY` byte-for-byte.
`LOGGER_HMAC_SECRET`	string	Outbound payload-signing secret for logger events. Must match its `LOGGER_HMAC_SECRET` byte-for-byte.

Optional

Variable	Default	Description
`OPENAGENT_FRONTEND_URL`	—	Extra CORS origin for production deployments.
`OPENAGENT_LOG_LEVEL`	`INFO`	DEBUG | INFO | WARNING | ERROR | CRITICAL
`OPENAGENT_UPSTREAM_CONNECT_TIMEOUT`	`10.0`	Seconds to wait for TCP connect to openagent-infra.
`OPENAGENT_UPSTREAM_READ_TIMEOUT`	`none`	Seconds (or `none` for unbounded) for streaming reads.
`OPENAGENT_HEALTH_TIMEOUT`	`5.0`	Seconds for upstream /health probe.
`OPENAGENT_BIO_PATH`	`/app/src/prompt/bio.txt`	Override path to bio.txt inside the container.
`OPENAGENT_LOGGER_QUEUE_MAX_SIZE`	`1000`	Max pending events in LoggerClient queue before drop-oldest.

Intentionally absent: `OPENAGENT_DEFAULT_REASONING_EFFORT`. `openagent-api` is pure pass-through; `openagent-infra`'s `REASONING_EFFORT` env var is the single source of truth for the default.

`OPENAGENT_INFRA_URL` by deployment topology

Scenario	Value

Scenario	Value
Everything on host, no Docker	`http://localhost:8002`
openagent-api in Docker, openagent-infra on host	`http://host.docker.internal:8002`
Both in Docker on a shared external network	`http://openagent-infra:8002`
External openagent-infra deployment	`https://infra.your-domain.com`

`LOGGER_URL` by deployment topology

Scenario	Value
Everything on host, no Docker	`http://localhost:8003`
openagent-api in Docker, openagent-logger on host	`http://host.docker.internal:8003`
Both in Docker on a shared external network	`http://openagent-logger:8003`
External openagent-logger deployment	`https://logger.your-domain.com`

Both services attach to the shared `openagent-network` (created and owned by `openagent-logger`) in the shared-network topology, so container-name addressing Just Works.

Security Model

This section documents the system-wide security posture for `openagent-api` and its position within the larger OpenAgent credential architecture. It complements the per-key descriptions in the ownership sections above; those describe each key in isolation, this describes how they fit together.

Pattern: compartmentalization (least privilege for credentials)

The OpenAgent system uses **compartmentalization** (least privilege applied to credentials) for all service-to-service authentication. Each service holds only the secrets for the boundaries it directly touches; no key is forwarded or relayed unchanged through the chain. This is the standard pattern for multi-service architectures and is what bounds the blast radius of any single-service compromise.

The contrast — a **bearer token relay** / **shared-secret architecture** — uses one secret end-to-end, forwarded by each service to the next. That pattern is operationally simpler but catastrophic in compromise, because stealing the key from any one service grants the same access as stealing it from all of them. `openagent-frontend`, `openagent-api`, `openagent-infra`, and `openagent-logger` all reject this pattern.

The logger boundary additionally rejects a weaker variant — **transport-only auth** (`X-API-Key` alone). Transport keys gate the wire boundary but provide no integrity guarantee for stored data. The HMAC signature stored on every row solves this: anyone with `LOGGER_HMAC_SECRET` can verify event integrity offline. That property survives forever; transport keys can rotate without invalidating it.

Per-service key inventory

Service	Holds	Does not hold

Service	Holds	Does not hold
openagent-frontend	OPENAGENT_API_KEY (outbound to openagent-api)	INFRA_API_KEY, LOGGER_API_KEY, LOGGER_HMAC_SECRET, PROVIDER_API_KEY
openagent-api	OPENAGENT_API_KEY (inbound), INFRA_API_KEY (outbound to infra), LOGGER_API_KEY (outbound transport to logger), LOGGER_HMAC_SECRET (outbound payload signing)	PROVIDER_API_KEY
openagent-infra	INFRA_API_KEY (inbound, named API_KEY on its side), PROVIDER_API_KEY (outbound to provider)	OPENAGENT_API_KEY, LOGGER_API_KEY, LOGGER_HMAC_SECRET
openagent-logger	LOGGER_API_KEY (inbound transport), LOGGER_HMAC_SECRET (inbound verification + stored on every row)	OPENAGENT_API_KEY, INFRA_API_KEY, PROVIDER_API_KEY
BYOC provider	PROVIDER_API_KEY (inbound)	None of the others

The keys are independent values, not derivations of one master secret. openagent-api is the only service holding two secrets for the same boundary — LOGGER_API_KEY and LOGGER_HMAC_SECRET both protect the api ↔ logger boundary, with different rotation profiles documented below.

Key generation

```
# OPENAGENT_API_KEY and INFRA_API_KEY (256 bits, hex)
python -c "import secrets; print(secrets.token_hex(32))"

# LOGGER_API_KEY and LOGGER_HMAC_SECRET (URL-safe base64; the logger's
convention)
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Each key is generated independently. Do not reuse one key as another, even temporarily. Do not use UUIDs, timestamps, or hand-typed strings. Placeholder values are acceptable only for "is the wiring connected" smoke tests against an isolated localhost stack; rotate to real values before the stack reaches any host another person could access, before any billable provider spend is incurred, and before any external party has visibility into the codebase or environment.

Blast-radius analysis

Compromised service	Keys exposed	Boundaries reachable	Boundaries protected
openagent-frontend	OPENAGENT_API_KEY	frontend → openagent-api	api → infra; api → logger; infra → provider

Compromised service	Keys exposed	Boundaries reachable	Boundaries protected
`openagent-api`	`OPENAGENT_API_KEY`, `INFRA_API_KEY`, `LOGGER_API_KEY`, `LOGGER_HMAC_SECRET`	frontend → api; api → infra; api → logger (incl. forging events with valid HMAC)	infra → provider. Pre-compromise rows in the logger stay HMAC-verifiable as long as `LOGGER_HMAC_SECRET` is rotated post-compromise.
`openagent-infra`	`INFRA_API_KEY`, `PROVIDER_API_KEY`	api → infra; infra → provider	frontend → api; api → logger. Capture layer keeps recording; infra's compromise becomes visible in the logger's `upstream_error` events.
`openagent-logger`	`LOGGER_API_KEY`, `LOGGER_HMAC_SECRET` (and DB access if the host itself is compromised)	reads captured `conversation_captures` (raw input/output text); could submit forged events the logger would accept	frontend → api; api → infra; infra → provider. Compromise bounded to the capture layer.
BYOC provider	`PROVIDER_API_KEY`	(provider's internal exposure)	All OpenAgent boundaries

In every scenario, compromise stops at the layer below the compromised service. There is no single key whose theft compromises the entire chain.

Service-to-service vs user-to-service auth

All keys above authenticate **services to services**, not users to services. There is no concept of user identity at the `openagent-api` layer — `OPENAGENT_API_KEY` is a shared secret between the frontend and the gateway, and any client holding it gets full access. This is a deliberate scoping decision for a reference implementation. The practical implication: anyone with deployment access to `openagent-frontend` (or who can read its `.env`) can use the system fully. Acceptable for solo development and trusted internal use; not acceptable for public exposure without an auth layer in front.

Key rotation procedure

Each rotation updates the secret in both services that share the boundary, then restarts both.

Rotating `OPENAGENT_API_KEY` (frontend ↔ api): generate a new value, update `openagent-api/.env` and `openagent-frontend/.env` with the same value, restart both containers, verify with `curl -H "X-API-Key: <new>" http://localhost:8001/health` → `{"status":"ok",...}`. The old key now returns 401.

Rotating `INFRA_API_KEY` (api ↔ infra): generate a new value, update `openagent-api/.env` and `openagent-infra/.env` (as its `API_KEY`) with the same value, restart both, verify `openagent-api`'s `/health` still reports `openagent_infra.status: ok`. If it becomes `unreachable`, the keys are out of sync.

Rotating `LOGGER_API_KEY` (api ↔ logger transport): generate via `token_urlsafe(48)`, update `openagent-logger/.env` and `openagent-api/.env`, restart both, verify by sending a `/chat` and

checking `openagent-logger:/stats` row counts increment. **This rotation is cheap** — the transport key only gates wire access; existing rows are unaffected. Rotate freely on a calendar cadence or on exposure.

Rotating `LOGGER_HMAC_SECRET` (api ↔ logger signing): generate via `token_urlsafe(48)`, update both `.env`s, restart both, verify the same way. **This rotation is expensive** — the signature is stored on every row, and pre-rotation signatures cannot be re-verified with the new secret. Rotate only on known compromise; if you must, document the cutover timestamp so anyone re-verifying knows which secret applies to which rows.

Rotating `PROVIDER_API_KEY` (infra ↔ provider): owned by `openagent-infra`; `openagent-api` is not involved.

When to rotate

Suspected exposure (a `.env` committed, copied, or shared insecurely): rotate immediately. Device compromise (a machine that held keys is lost or stolen): rotate every key it held. Environment promotion (dev → staging → prod): always rotate; never let one secret span environments. Calendar cadence: routine rotation as a precaution. `LOGGER_HMAC_SECRET` is the exception — rotate only on known compromise, never routinely.

Storage tiers

A general hierarchy of secret-storage practice, with where this reference sits and where a production deployment should move:

- **`.env` files on disk, gitignored** — current state, appropriate for local development. Risk: accidental commit or copy. Mitigation: `.gitignore` excludes `.env`; a pre-commit hook blocking `.env*` is a good addition.
- **Container orchestration secrets** — appropriate before any remote deployment. Docker Compose `secrets:`, Docker Swarm `docker secret`, Kubernetes `Secret` with encryption-at-rest. Scoped to specific services rather than visible to anyone who can read the host filesystem.
- **Dedicated secrets manager** — appropriate for any serious production use. AWS Secrets Manager, HashiCorp Vault, GCP Secret Manager, Azure Key Vault, Doppler. Secrets fetched at startup using the service's own identity; never on disk. Adds audit logging and rotation primitives natively.
- **Workload identity / mTLS** — no shared secrets at all; mutual TLS with per-service certificates or SPIFFE/SPIRE-style attestation.

Two non-negotiables regardless of tier: `.env` is never committed; placeholder values never reach a non-localhost stack.

Transit security

- **TLS for any non-localhost transit.** Transport keys travel in HTTP headers. On `localhost`/`host.docker.internal`, traffic doesn't leave the host and plaintext HTTP is acceptable. Any deployment where a service runs on a separate host requires TLS end-to-end across all four boundaries. Terminate TLS at a public-facing reverse proxy; use mTLS or VPC-internal traffic between backend services. HMAC signatures travel in the JSON body and add a payload-integrity layer on top of TLS — complementary, not a substitute.
- **Headers, never URLs.** Transport keys are sent as `X-API-Key` headers (or `Authorization: Bearer` for the provider). Never query strings — those are logged by web servers, proxies, browsers, and bug trackers.
- **Never echoed.** `openagent-api` does not log `INFRA_API_KEY`, `LOGGER_API_KEY`, or `LOGGER_HMAC_SECRET`, and does not include them in error or `/health` responses.

Frontend-supplied system-message rejection (defense-in-depth)

A second defense layer separate from the auth chain: `openagent-api` drops any `"system"`-role messages in the `/chat` body, regardless of authentication. The canonical persona is `bio.txt` and only `bio.txt`. This protects against a tampered or out-of-date frontend that holds a valid `OPENAGENT_API_KEY` but attempts to override the identity. Auth alone cannot prevent prompt-override attacks; message-shape filtering does.

Container / Deployment

Image

- **Base:** `python:3.11-slim`
- **Tag:** `openagent-api:1.0.0`
- **Container name:** `openagent-api`
- **Size (approximate):** ~150–200 MB (pure Python, no CUDA, no BLAS)
- **User:** non-root (`openagent`, uid 1000)

Build

```
# From repo root
docker-compose up -d --build
```

The `--build` flag is **required** when `src/backend/api.py`, `src/client/infra.py`, `src/client/logger.py`, `src/prompt/bio.txt`, `requirements.txt`, or the Dockerfile change.

Port mapping

- **Host port 8001 → Container port 8001**
- uvicorn binds to `0.0.0.0:8001` inside the container (per the Dockerfile CMD)

Volumes

None by default. The container is stateless; `bio.txt` is baked in at build time. For development, optionally mount `./src/prompt:/app/src/prompt:ro` to live-edit the persona without rebuilding.

Restart policy

`unless-stopped`.

Healthcheck

Defined in the Dockerfile via `HEALTHCHECK`. Probes `/health` every 30 seconds with the `X-API-Key` header from `OPENAGENT_API_KEY`. Marks the container unhealthy after three consecutive failures. The healthcheck does NOT validate the logger boundary — logger unavailability is non-fatal (fire-and-forget) and including it would couple `openagent-api`'s reported health to a non-essential dependency.

File Structure

```
openagent-api/
├── docker/
│   └── api/
│       └── Dockerfile              # Python 3.11 slim + non-root +
healthcheck
├── src/
│   ├── backend/
│   │   └── api.py                  # The FastAPI app (single file)
│   ├── client/
│   │   ├── __init__.py             # Package marker
│   │   ├── infra.py                # InfraClient: streaming /chat + /health
proxy to openagent-infra
│   │   └── logger.py               # LoggerClient: fire-and-forget emitter
to openagent-logger
│   └── prompt/
│       └── bio.txt                 # Persona (baked into image)
├── docs/
│   └── DATASHEET.md                # This document
├── docker-compose.yml              # Single-service compose
├── requirements.txt                # fastapi, uvicorn, httpx, python-dotenv
├── .env                            # secrets — never committed
├── .env.example                    # template for .env
├── .dockerignore
├── .gitignore
└── README.md
```

The runtime is three Python files (`backend/api.py` + `client/infra.py` + `client/logger.py`) plus the persona text and the deployment shell. Dependency footprint is deliberately tiny: **fastapi, uvicorn, httpx, python-dotenv** and nothing else. Both `InfraClient` and `LoggerClient` use only the stdlib (`hmac`, `hashlib`, `json`, `asyncio.Queue`) plus the already-present `httpx` — no extra pip packages. The dependency direction is explicit: backend depends on client; client never depends on backend.

Integration Notes for Other Services

For openagent-frontend

`openagent-frontend` is the primary client. It:

- Points its backend URL config at `OPENAGENT_API_URL`.
- Carries no persona; sends only user/assistant turns (no `system` message).
- Optionally includes `reasoning_effort` on the request body (or omits it to let `openagent-infra` default), and can expose it as a UI toggle (Quick / Standard / Deep).
- Holds `OPENAGENT_API_KEY` and sends it as `X-API-Key` on `/chat` and `/health`.
- Polls `/health` to gate the chat input; the top-level `status` vocabulary (`ok` / `loading` / `unreachable`) is what its gate-open logic reads.
- Decodes each SSE event as a JSON ChatCompletion chunk, routing `choices[0].delta.reasoning` to the reasoning surface and `choices[0].delta.content` to the chat bubble; the stream terminates with an empty-delta `finish_reason: "stop"` chunk followed by `data: [DONE]`.

For openagent-infra

`openagent-api` is the only client of `openagent-infra`. It opens a streaming POST to `OPENAGENT_INFRA_URL/chat` with `X-API-Key: INFRA_API_KEY`, prepending `bio.txt` as the system message. It never sets the `model` field, so every call routes to the base model. It proxies `openagent-infra`'s `/health` and translates `degraded` → `loading` for the frontend.

For openagent-logger

`openagent-api` emits five event types per `/chat` call via the in-process `LoggerClient`, each HMAC-signed and stored with its signature for offline re-verification. `openagent-logger` owns event storage, partitioning, retention, replay-window enforcement, and the `/stats` and `/health` endpoints. `openagent-api` owns event GENERATION and the wire-side signing. The two MUST agree byte-for-byte on the envelope shape, payload schemas, and canonical-string protocol; a change on either side requires a coordinated release. The contract is fire-and-forget — no client-side retry, no backoff, no dead-letter queue — so a logger outage causes WARNING log lines but no `/chat` impact.

Design Decisions

Why is the persona owned by openagent-api, not the frontend?

Identity is product-layer **backend** logic. The frontend is replaceable; the identity is not. Keeping `bio.txt` server-side means any client (mobile, CLI, alternate UI) shares one canonical agent, and a tampered or out-of-date frontend cannot override the persona.

Why independent secrets at each boundary instead of one shared secret?

Isolation of compromise. With one shared key end-to-end, compromise of any service compromises all of them. With independent secrets at each boundary, each compromise is isolated to its layer (see the blast-radius table). All keys have 256+ bits of entropy; none is brute-forceable individually. The benefit is structural, not entropy.

Why two secrets for openagent-logger (LOGGER_API_KEY + LOGGER_HMAC_SECRET)?

Different threat models and rotation profiles. `LOGGER_API_KEY` (transport) gates wire access; rotation is cheap (existing rows unaffected). `LOGGER_HMAC_SECRET` (payload signing) provides integrity — the stored signature lets a consumer verify any row offline without trusting the original transport; rotation is expensive (pre-rotation signatures can't be re-verified). Splitting them lets you rotate the transport key on a cadence without disturbing the integrity guarantee on stored data.

Why fire-and-forget logger emission?

The `/chat` hot path must not block on the capture layer. A synchronous emit-per-event design would add round-trip latency per chat turn AND couple `openagent-api`'s reliability to the logger's. Fire-and-forget makes emit methods synchronous, no-I/O, microsecond-latency; a background task drains in parallel. Logger unavailability is invisible to the frontend. The trade-off is event loss when the logger is down — accepted for a reference implementation.

Why an in-memory queue instead of a durable outbox?

Zero infrastructure: no new dependencies, no new services, no new failure modes. It composes naturally with the FastAPI lifespan and the existing httpx client. The cost is that events queued in memory at the moment of a

container restart are lost. A durable outbox would survive restarts at the cost of a new dependency, schema, and failure mode; for a reference implementation, in-memory is the right trade.

Why drop-oldest queue overflow (not drop-newest)?

During a sustained logger outage, fresher data is more useful than stale data when the storm passes. Drop-oldest also pairs cleanly with the logger's 300-second replay window — the oldest events would be the most likely to be rejected as stale on any replay anyway.

Why no reasoning chain capture?

`conversation_capture.output_text` contains only `delta.content` tokens (the visible answer), not `delta.reasoning` (the chain-of-thought). The reasoning format is model-specific and likely to change; the capture is meant to record user-facing behavior — the answers actually given — not the model's internal reasoning style. This is reversible: if reasoning capture becomes valuable, a new optional `reasoning_text` field can be added in a coordinated api + logger release.

Why is `reasoning_effort` pure pass-through with no openagent-api default?

The setting has one source of truth: `openagent-infra`'s `REASONING_EFFORT` env var. A second default here would mean two places to check when debugging, and they could disagree.

Why SSE relay instead of buffering and returning JSON?

Generations take seconds to minutes (plus provider cold-start). Buffering means a multi-minute spinner with no feedback. Streaming is the only viable UX for this latency profile.

Why forward bytes instead of parse-and-re-emit?

Anything `openagent-api` parses on the relay path, it can break. Forwarding raw bytes means the frontend's parser keeps working unmodified and the relay stays simple. The cost is that mid-stream upstream errors must be surfaced as in-band SSE events rather than HTTP status codes, but the frontend handles that cleanly. The v side-channel parser does parse the same stream, but in parallel (downstream of the yield) and only to extract `delta.content`; parse failures there are silently tolerated and never affect the relay.

Why is `/health` authenticated?

It reveals operational state — whether the upstream is reachable, what version is running, whether the provider worker is warm. That's internal information. Auth adds zero friction for the frontend (it has the key) and keeps random scanners out.

Why drop frontend-supplied system messages instead of rejecting?

Robust over strict. During a transition where an older frontend may still send a system message, dropping (with a warning log) keeps the request working; the operator sees the warning and updates the frontend.

Why httpx instead of requests or aiohttp?

`httpx` is async-native (FastAPI's event loop can `await` it), supports streaming exactly as the SSE pump needs (`aiter_raw()`), and has a familiar API. `requests` is sync and would block the event loop. Both `InfraClient` and `LoggerClient` reuse the same `httpx` library, each with a separate `AsyncClient` instance.

Why two separate httpx.AsyncClient instances?

The infra and logger boundaries differ on every axis: headers (`INFRA_API_KEY` vs `LOGGER_API_KEY`), base URLs, timeout profiles (infra: 10s connect, unbounded read for cold-start; logger: 5s connect, 10s read for fail-fast), and connection-pool semantics (long-lived streams vs many short POSTs). Sharing one client would force compromises on all of them. Each client lives in its own module under `src/client/`, with explicit dependency direction (backend → client, never the reverse).

Why a non-root user in the container?

A container escape that gives root inside should not give root outside. The `openagent` user (uid 1000) is created in the Dockerfile and the runtime drops privilege before uvicorn starts.

Why is the infra read timeout unbounded?

The first request after a serverless worker has scaled to zero waits for the worker to spin up; even on a warm worker, `high` reasoning effort can run for minutes. A finite read timeout would kill long but legitimate generations. Connect timeout stays short (10s) so an unreachable upstream fails fast. The logger boundary uses short timeouts (5s/10s) because fire-and-forget emissions should fail fast.

Why python:3.11-slim?

Matches `openagent-frontend`'s base image for consistency. Pure-Python service, no GPU stack — slim is the right tier.

Why a single-file FastAPI app (api.py) plus a client package?

The service is one HTTP gateway with two endpoints and two dedicated outbound client wrappers (`InfraClient`, `LoggerClient`) kept separate from `api.py` to delimit concerns cleanly. Splitting `api.py` further into routers/services/schemas folders would be premature abstraction at this size. The `src/client/` layout is the right amount of separation: backend depends on client; client never depends on backend.

Why port 8001?

Port convention: 8000 = frontend (user-facing), 8001 = api (the API the frontend calls), 8002 = infra (model layer, called only by api), 8003 = logger (capture layer, called only by api), provider = remote inference (called only by infra). The numbering reflects the request flow.

Known Limitations

Stateless across requests

No conversation history at this layer. The frontend re-sends the full message list on every turn, and `openagent-api` forwards it. There is no persistence.

Single shared inbound key

`OPENAGENT_API_KEY` is one secret for one client. There is no concept of "user A vs user B." `openagent-api` emits `user_id: null` on every event.

No rate limiting

The gateway trusts authenticated clients absolutely. If exposed beyond the OpenAgent product, rate limiting belongs at a reverse proxy.

Event capture is best-effort

The capture pipeline is fire-and-forget with an in-memory queue, so events can be lost when the logger is unreachable (drop after a WARNING) or when the queue fills during a sustained outage (drop-oldest). `/chat` is unaffected by this. The captured events also do NOT include per-user attribution (`user_id` always null), validated session correlation (`session_id` always null), or the reasoning chain (`output_text` is `delta.content` only).

Context window truncation deferred

`openagent-api` forwards whatever messages the frontend sends. Long enough conversations will eventually 400 from `openagent-infra` once the model's context window is exceeded. There is no truncation or summarisation at this layer.

Cold-start latency inherited from the provider

Serverless workers scale to zero when idle; the first request after inactivity waits for the worker to spin up. `openagent-api` correctly reports `loading` during this time, but cannot make the worker spin up faster. Warm-path requests respond in seconds.

Reasoning-format display variance

Some upstream serving stacks emit reasoning-format delimiters that can occasionally bleed into the stream, depending on the runtime's parser support. `openagent-api` forwards bytes byte-for-byte and does not normalise this — display is the frontend's concern. The side-channel parser depends on the same chunk shape to extract `delta.content`; a parser change there would also need to track upstream format changes.

Mid-stream upstream errors are in-band

Once SSE headers go out (HTTP 200), `openagent-api` can no longer change the status code. Mid-stream upstream failures are surfaced as `data: [ERROR ...]\n\n` followed by `data: [DONE]\n\n`; the HTTP status stays 200. The corresponding `upstream_error` ops_event captures the operational signal regardless.

bio.txt requires a rebuild

The persona is baked into the image at build time. Rotating it requires `docker-compose up -d --build` (or a dev volume mount over `/app/src/prompt`).

---

openagent-api — part of the OpenAgent system